

# Guía de Diseño v3.0

## Valdris: El Núcleo Profundo

Documento operativo de implementación · Complemento a la Guía de Proyecto v3

|           |                                                                        |
|-----------|------------------------------------------------------------------------|
| Proyecto  | Valdris: El Núcleo Profundo — Juego por Turnos JavaFX                  |
| Grupo     | H12GEXTRA — Marcos Castro · Ventura Pacheco · Marino Rodríguez         |
| Versión   | v3.0 — Punto 2 añadido: IA enemiga completa con pseudocódigo           |
| Compañero | Guía de Proyecto v3.0 (narrativa, arquitectura, advertencias técnicas) |

### Cambios respecto a v2.0

- Punto 2 añadido: IA completa de los 8 subtipos de enemigo con stats, pseudocódigo y notas de implementación.
- Sistema de movimiento cardinal confirmado: BFS 4 direcciones, diagonal cuesta 2 pasos naturalmente.
- Sistema de efectos de estado definido: LSE en Unit, duración fija de 2 turnos.
- Línea de visión (Bresenham) definida para Archer y Mage — ambos requieren visión directa para atacar.
- Tabla resumen de todos los enemigos al final del Punto 2.

---

## Punto 1 — Sistema de Turnos Completo

---

Referencia definitiva para implementar TurnManager. Ver v2.0 para el razonamiento completo de cada decisión.

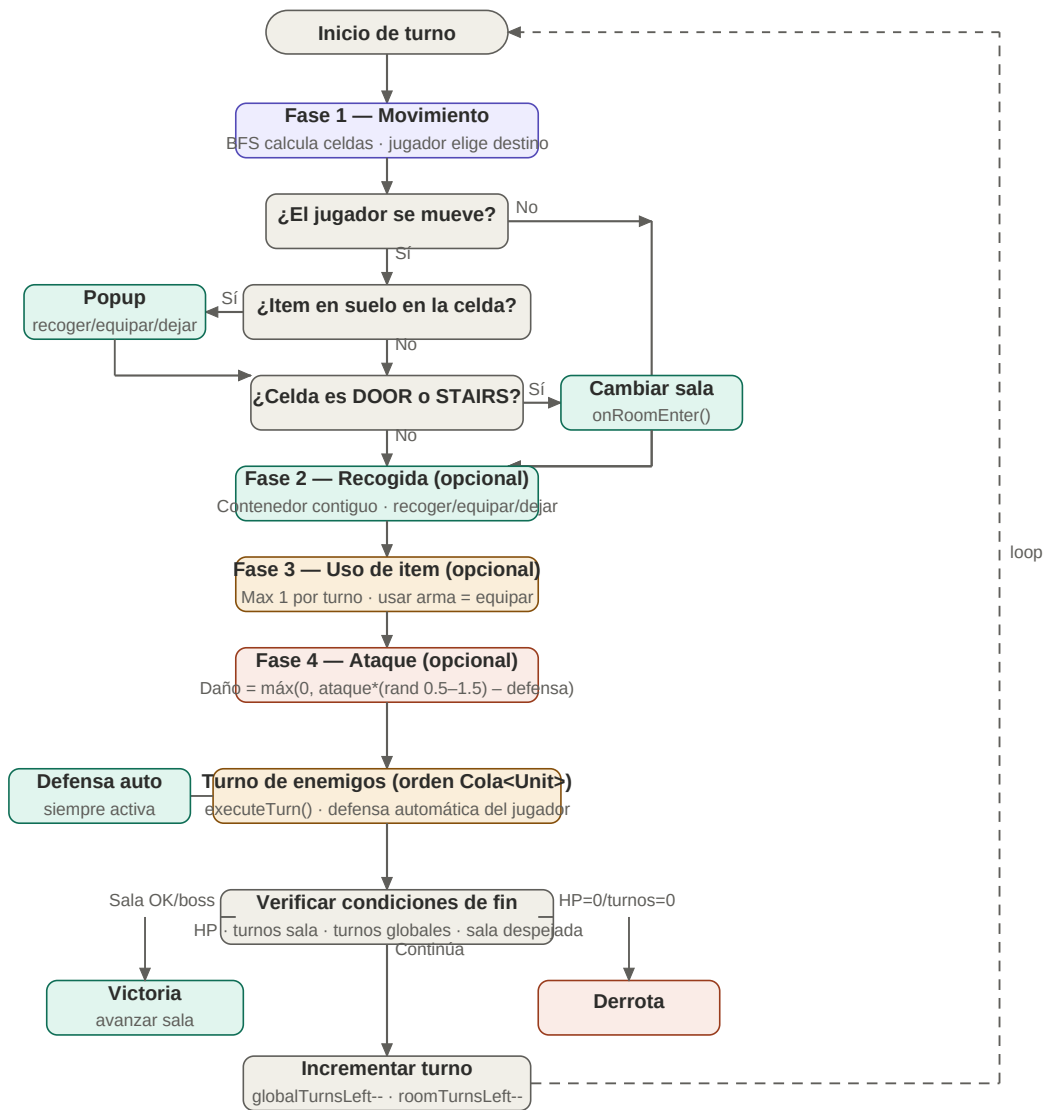
### 1.0 Diagrama de flujo completo de un turno

Flujo completo: cuatro fases del jugador en orden fijo, turno de enemigos, verificación de condiciones y loop. Las flechas punteadas representan el loop al inicio del siguiente turno.

■ FASE JUGADOR

■ FASE ENEMIGOS

■ FIN DE TURNO



■ Fase jugador

■ Evento especial

■ Turno enemigos

■ Combate/derrota

■ Control flujo

## 1.1 Acciones disponibles del jugador

| Acción      | Máx/turno | Restricción                                   | Notas                                                        |
|-------------|-----------|-----------------------------------------------|--------------------------------------------------------------|
| Movimiento  | 1         | Siempre primero                               | BFS 4 direcciones cardinales. Diagonal = 2 pasos BFS.        |
| Recogida    | 1         | Fase 1 (suelo) o Fase 2 (contenedor contiguo) | Item suelo: popup automático. Contenedor: acción manual.     |
| Uso de item | 1         | Solo inventario. Usar arma = equipar          | No se puede usar poción Y atacar con arma en el mismo turno. |
| Ataque      | 1         | Enemigo en rango BFS del arma equipada        | Sin arma: rango 1, daño base.                                |
| Ceder turno | —         | En cualquier momento                          | No hace nada. Pasa al turno enemigo.                         |

### Caso especial — equipar vs atacar

- Usar arma del inventario = equiparla (Fase 3). NO es atacar.
- Atacar con arma ya equipada SÍ es ataque (Fase 4).
- Se puede recoger una espada y atacar con ella en el mismo turno.

## 1.2 Fórmula de combate

### Fórmula oficial de daño

- $\text{vida\_defensor} = \text{vida\_defensor} - \text{máximo}(0, \text{ataque} * \text{aleatorio} - \text{defensa})$
- $\text{aleatorio} = \text{Math.random()} * 1.0 + 0.5 \rightarrow \text{rango } [0.5, 1.5]$
- $\text{ataque} = \text{ataque base} + \text{modificadores arma} + \text{otros}$
- $\text{defensa} = \text{defensa base} + \text{modificadores armadura} + \text{otros}$

```
public int calcularDanio(Unit atacante, Unit defensor) {
    double aleatorio = Math.random() * 1.0 + 0.5; // [0.5, 1.5]
    int ataque = atacante.getAtaqueTotal();
    int defensa = defensor.getDefensaTotal();
    return Math.max(0, (int)(ataque * aleatorio) - defensa);
}
```

## 1.3 Walkability y movimiento cardinal

| Condición  | Transitable                                              | Bloqueado                                |
|------------|----------------------------------------------------------|------------------------------------------|
| Tipo celda | FLOOR, DOOR, DOOR_HIDDEN(activa),<br>RUNE, LEVER, STAIRS | WALL, DOOR_LOCKED                        |
| Contenido  | unit==null Y (item==null O !requiresAdjacent)            | unit!=null O item.requiresAdjacent==true |

```
public boolean isWalkable() {
    if (type==WALL || type==DOOR_LOCKED) return false;
    if (unit != null) return false;
}
```

```

if (item != null && item.isRequiresAdjacent()) return false;
return true;
}

```

## 1.4 Contadores y condiciones de fin

| Condición                            | Resultado                | Quién detecta               |
|--------------------------------------|--------------------------|-----------------------------|
| HP jugador <= 0                      | Derrota                  | TurnManager tras takeDamage |
| globalTurnsLeft == 0                 | Derrota                  | TurnManager fin de turno    |
| roomTurnsLeft == 0 (si hasRoomTimer) | Derrota                  | TurnManager fin de turno    |
| Todos los enemigos de sala muertos   | Sala despejada           | TurnManager tras onDeath    |
| ParasitoEnemy muerto (Zona 5)        | Victoria — triggerEnding | TurnManager                 |

## 1.5 División de trabajo

| Miembro | Bloque               | Qué incluye                                                                                          |
|---------|----------------------|------------------------------------------------------------------------------------------------------|
| Marino  | Motor + integración  | Modelo completo (Fase 1), lógica (Fase 2: BFS, TurnManager, combate, acertijos), integración JavaFX. |
| Pacheco | Persistencia + tests | Fase 3 (LectorJSON, GameState, serialización plana). Tests JUnit desde el día 1.                     |
| Castro  | Interfaz JavaFX      | Fase 4: GridPane, paneles, selección personaje, diálogos, finales. Empieza día 10.                   |

---

## Punto 2 — IA Enemiga por Tipo y Subtipo

---

Esta sección define el comportamiento exacto de cada tipo y subtipo de enemigo: sus stats, la lógica de su `executeTurn()` en pseudocódigo, y las notas de implementación clave. Es la referencia para implementar cada clase de `Enemy` concreta.

### Principios generales de la IA

- Todas las IAs usan BFS real sobre `Cell[][]` para calcular distancias — más preciso que Manhattan.
- El resultado del BFS del jugador se cachea al inicio del turno de enemigos y se reutiliza para todos — 1 BFS por turno, no N.
- El método `BFSMovimiento.getCellsInRange(row, col, range, room)` se reutiliza tanto para movimiento como para calcular celdas en rango de ataque.
- Movimiento estrictamente cardinal (4 direcciones). La diagonal cuesta 2 pasos BFS naturalmente — no hay caso especial.
- Línea de visión requerida para atacar: Archer, Francotirador, Destructor, Controlador. Warrior, Berserker y Guardián no la necesitan.

### 2.1 Línea de visión — algoritmo de Bresenham

Los enemigos de rango (Archer, Mage) solo pueden atacar si tienen línea de visión directa al jugador — no disparan a través de paredes. Se usa el algoritmo de Bresenham para trazar la línea entre atacante y objetivo celda a celda.

```
public boolean tieneLineaDeVision(int r1, int c1, int r2, int c2, Room room) {  
    // Bresenham sobre la cuadrícula de la sala  
    int dr = Math.abs(r2-r1), dc = Math.abs(c2-c1);  
    int sr = r1<r2 ? 1 : -1, sc = c1<c2 ? 1 : -1;  
    int err = dr - dc;  
    int r = r1, c = c1;  
    while (r != r2 || c != c2) {  
        if (r != r1 || c != c1) { // no comprobar celda del atacante  
            Cell cell = room.getCell(r, c);  
            if (cell.getType() == CellType.WALL) return false;  
        }  
        int e2 = 2 * err;  
        if (e2 > -dc) { err -= dc; r += sr; }  
        if (e2 < dr) { err += dr; c += sc; }  
    }  
    return true;  
}
```

### 2.2 Sistema de efectos de estado

El Mage Controlador aplica efectos de estado al jugador. Los efectos se gestionan mediante una LSE en la clase Unit, decrementada al inicio de cada turno del jugador.

```
// Efecto.java
public class Efecto implements Comparable<Efecto> {
    public enum TipoEfecto { SLOW, BLIND, CURSE }
    private TipoEfecto tipo;
    private int turnosRestantes;
}

// Al inicio del turno del jugador en TurnManager:
public void decrementarEfectos(Player player) {
    LSE<Efecto> efectos = player.getEfectosActivos();
    for (int i = 0; i < efectos.getSize(); i++) {
        Efecto e = efectos.get(i);
        e.decrementarTurno();
        if (e.getTurnosRestantes() == 0) efectos.del(e);
    }
}
```

| Efecto               | Duración | Impacto                                                                       | Implementación                                                                     |
|----------------------|----------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| SLOW<br>(Ralentizar) | 2 turnos | movePoints = Math.max(1, original/2) redondeado arriba                        | Player.setMovePointsModificado(). Al expirar: restaurar valor base.                |
| BLIND<br>(Ceguera)   | 2 turnos | movePoints = Math.ceil(original / 2.0) — reduce a la mitad redondeando arriba | Mismo mecanismo que SLOW. JavaFX no muestra el resaltado BFS mientras está activo. |
| CURSE<br>(Maldición) | 2 turnos | Daño recibido × 1.5 durante 2 turnos                                          | En calcularDanio(): if player.tieneEfecto(CURSE) → daño *= 1.5                     |

#### Ceguera — ejemplos por personaje

- Kael (movePoints=3):  $\text{Math.ceil}(3/2.0) = 2 \rightarrow$  queda con 2 movePoints.
- Syra (movePoints=5):  $\text{Math.ceil}(5/2.0) = 3 \rightarrow$  queda con 3 movePoints.
- Dorath (movePoints=2):  $\text{Math.ceil}(2/2.0) = 1 \rightarrow$  queda con 1 movePoint.

## 2.3 Familia Warrior

### Warrior

Subtipo: Base — Warrior

Enemigo de cuerpo a cuerpo estándar. Persigue al jugador por el camino más corto usando BFS y ataca si queda adyacente tras el movimiento. Simple y predecible — el jugador aprende a esquivarlo rápido, lo que hace a los subtipos más sorprendentes.

| Stat       | Valor | Stat         | Valor |
|------------|-------|--------------|-------|
| movePoints | 2     | attackRange  | 1     |
| damage     | 15    | defense      | 8     |
| HP base    | 40    | línea visión | No    |

```
executeTurn(player, room):
    camino = BFS(miPos, playerPos, room)
    mover min(movePoints, camino.size()-1) pasos
    si distBFS(miPos, playerPos) == 1:
        atacar(player) // adyacente = ataca
```

### Berserker

Subtipo: Subtipo — Warrior

Más rápido y agresivo que el Warrior base pero muy frágil. Prioriza llegar al jugador a toda costa ignorando el posicionamiento óptimo. Fácil de matar si el jugador mantiene distancia, peligroso si lo deja acercarse. El Invocador genera Berserkers.

| Stat       | Valor | Stat         | Valor |
|------------|-------|--------------|-------|
| movePoints | 4     | attackRange  | 1     |
| damage     | 13    | defense      | 3     |
| HP base    | 25    | línea visión | No    |

```
executeTurn(player, room):
    camino = BFS(miPos, playerPos, room)
    // Ignora posicionamiento, va directo
    mover min(movePoints, camino.size()-1) pasos
    si distBFS(miPos, playerPos) == 1:
        atacar(player)
```

■ Invocado por el Mage Invocador. Stats reducidos para no sobrecargar la sala.



**Guardián**

*Subtipo: Subtipo — Warrior*

No persigue al jugador libremente. Defiende una zona fija (su celda de spawn). Si el jugador se acerca a radio BFS  $\leq 3$ , se mueve 1 celda hacia él. Si el jugador se aleja, vuelve a guardPosition. Alto daño y altísima defensa — el jugador tiene que decidir si merece la pena enfrentarlo.

| Stat       | Valor | Stat         | Valor |
|------------|-------|--------------|-------|
| movePoints | 1     | attackRange  | 1     |
| damage     | 15    | defense      | 18    |
| HP base    | 60    | línea visión | No    |

```
executeTurn(player, room):
    dist = distBFS(miPos, playerPos, room)
    si dist <= 3:
        camino = BFS(miPos, playerPos, room)
        mover 1 paso hacia player
        si distBFS(miPos, playerPos) == 1: atacar(player)
    sino:
        si miPos != guardPosition:
            mover 1 paso hacia guardPosition // vuelve
```

■ guardPosition se asigna en spawn y nunca cambia. El Guardián nunca se aleja más de 3 celdas BFS de ella.

## 2.4 Familia Archer

### Archer

*Subtipo: Base — Archer*

Mantiene una zona de confort entre distancia mínima y máxima. Si el jugador se acerca demasiado, huye. Si está en rango con línea de visión, ataca sin moverse. Si el jugador está fuera de rango, se acerca. Requiere línea de visión para atacar.

| Stat         | Valor | Stat            | Valor |
|--------------|-------|-----------------|-------|
| movePoints   | 3     | attackRange     | 4     |
| damage       | 10    | defense         | 4     |
| HP base      | 30    | minComfortRange | 2     |
| línea visión | Sí    |                 |       |

```
executeTurn(player, room):
    dist = distBFS(miPos, playerPos, room)
    si dist < minComfortRange: // demasiado cerca
        mover en dirección opuesta al jugador
    sino si dist <= attackRange Y lineaVision(miPos, playerPos):
        atacar(player) // en zona cómoda con visión
    sino:
        camino = BFS(miPos, playerPos, room)
        mover min(movePoints, pasos) acercándose
```

■ La dirección opuesta se calcula invirtiendo el primer paso del BFS hacia el jugador.

## Francotirador

*Subtipo: Subtipo — Archer*

Mayor rango y daño que el Archer base pero dispara cada 2 turnos. Se posiciona lejos y espera. El jugador sabe que viene un disparo potente pero no puede evitar estar en rango fácilmente. El cooldown crea ritmo de tensión.

| Stat       | Valor    | Stat            | Valor |
|------------|----------|-----------------|-------|
| movePoints | 2        | attackRange     | 5     |
| damage     | 15       | defense         | 3     |
| HP base    | 28       | minComfortRange | 4     |
| cooldown   | 2 turnos | línea visión    | Sí    |

```
executeTurn(player, room):
  si cooldownRestante > 0:
    cooldownRestante--
    // Solo se mueve para posicionarse, no ataca
    posicionarse(player, room)
  sino:
    dist = distBFS(miPos, playerPos, room)
    si dist <= attackRange Y lineaVision(miPos, playerPos):
      atacar(player)
      cooldownRestante = 2
    sino: posicionarse(player, room)

posicionarse(): igual que Archer base (zona de confort minComfortRange=4)
```

■ El turno de cooldown no es desperdiciado — el Francotirador usa ese turno para mejorar posición.

## 2.5 Familia Mage

### Destructor

*Subtipo: Subtipo Mage — Opción A (Área)*

No se mueve nunca. Lanza un hechizo de área centrado en el jugador que daña todas las celdas en radio 2. Obliga al jugador a moverse constantemente para salir del área. No discrimina — también daña a otros enemigos cercanos al jugador. El jugador debe priorizar mantener distancia o eliminarlo rápido.

| Stat         | Valor                       | Stat        | Valor   |
|--------------|-----------------------------|-------------|---------|
| movePoints   | 0                           | attackRange | 5       |
| aoéRadius    | 2                           | damage      | 8/celda |
| defense      | 5                           | HP base     | 45      |
| línea visión | Sí — para el centro del AOE |             |         |

```
executeTurn(player, room):
    dist = distBFS(miPos, playerPos, room)
    si dist <= attackRange Y lineaVision(miPos, playerPos):
        // AOE centrado en el jugador
        para cada celda en room:
            si distManhattan(celda, playerPos) <= aoéRadius:
                si celda.getUnit() != null:
                    celda.getUnit().takeDamage(calcularDanio(this, celda.getUnit()))
        // Si fuera de rango: no hace nada ese turno
```

■ *distManhattan para el radio del AOE (más barato que BFS). La línea de visión se comprueba solo para el centro, no para cada celda del área.*

## Controlador

*Subtipo: Subtipo Mage — Opción B (Efectos de estado)*

Bajo daño directo pero aplica efectos de estado que disrumpen al jugador. Elige uno de tres efectos aleatoriamente al atacar: SLOW, BLIND o CURSE. El jugador necesita gestionar los efectos activos y priorizar eliminar al Controlador antes de que acumule demasiados efectos sobre él.

| Stat         | Valor | Stat           | Valor    |
|--------------|-------|----------------|----------|
| movePoints   | 2     | attackRange    | 3        |
| damage       | 4     | defense        | 4        |
| HP base      | 35    | effectDuration | 2 turnos |
| línea visión | Sí    |                |          |

```
executeTurn(player, room):
    dist = distBFS(miPos, playerPos, room)
    si dist <= attackRange Y lineaVision(miPos, playerPos):
        // Ataque con efecto aleatorio
        player.takeDamage(calcularDanio(this, player))
        efecto = elegirEfectoAleatorio() // SLOW, BLIND o CURSE
        player.addEfecto(new Efecto(efecto, 2))
        log('Controlador aplica ' + efecto + ' a ' + player.getName())
    sino:
        camino = BFS(miPos, playerPos, room)
        mover min(movePoints, pasos) hacia player
```

■ Si el jugador ya tiene el mismo efecto activo, se reinicia la duración (no se acumula). Evita situaciones injustas de stacking.

## Invocador

*Subtipo: Subtipo Mage — Opción C (Invocaciones)*

No ataca directamente. Genera presión creciente invocando Berserkers cada 2 turnos en celdas vacías adyacentes. El jugador debe decidir entre eliminar al Invocador primero (difícil, se aleja) o gestionar los Berserkers invocados. Diseñado para crear caos táctico.

| Stat         | Valor | Stat               | Valor    |
|--------------|-------|--------------------|----------|
| movePoints   | 2     | attackRange        | 0        |
| damage       | 0     | defense            | 6        |
| HP base      | 50    | invocationCooldown | 2 turnos |
| línea visión | No    |                    |          |

```
executeTurn(player, room):
    cooldownRestante--
    si cooldownRestante <= 0:
        celdas = getCeldasVaciasAdyacentes(miPos, radio=2, room)
        si celdas no vacía:
            // Celda más cercana al jugador
            mejor = celdaMasCercanaA(celdas, playerPos)
            berserker = new Berserker(mejor.row, mejor.col, room.getId())
            room.addEnemy(berserker)
            turnQueue.addEnd(berserker) // se une al orden de turnos
            cooldownRestante = 2
            // Siempre intenta acercarse al jugador para tener celdas cerca
            camino = BFS(miPos, playerPos, room)
            mover min(movePoints, pasos) hacia player
```

■ `getCeldasVaciasAdyacentes()` solo devuelve celdas con `isWalkable()==true` y sin unidad. El Berserker invocado actúa desde el siguiente turno.

### 2.6 Tabla resumen — todos los enemigos

| Enemigo       | Tipo         | Mov e | Rng atk | Daño | Def | HP | Especial                               |
|---------------|--------------|-------|---------|------|-----|----|----------------------------------------|
| Warrior       | Base Warrior | 2     | 1       | 15   | 8   | 40 | BFS hacia jugador                      |
| Berserker     | Sub Warrior  | 4     | 1       | 13   | 3   | 25 | Carga directa · invocado por Invocador |
| Guardián      | Sub Warrior  | 1     | 1       | 15   | 18  | 60 | Defiende zona fija radio 3             |
| Archer        | Base Archer  | 3     | 4       | 10   | 4   | 30 | Zona confort · línea visión            |
| Francotirador | Sub Archer   | 2     | 5       | 15   | 3   | 28 | Cooldown 2 turnos · rango alto         |
| Destructor    | Sub Mage A   | 0     | 5       | 8×   | 5   | 45 | AOE radio 2 · no se mueve              |
| Controlador   | Sub Mage B   | 2     | 3       | 4    | 4   | 35 | Efecto aleatorio 2 turnos              |
| Invocador     | Sub Mage C   | 2     | 0       | 0    | 6   | 50 | Invoca Berserkers cada 2 turnos        |

#### Aparición aleatoria por sala

- Cada sala define cuántos Warrior, Archer y Mage aparecen (número fijo por diseño de zona).
- El subtipo se elige aleatoriamente al generar la sala: Random.nextInt() selecciona entre los subtipos disponibles de cada familia.
- Zona 1: solo Warrior base y Archer base (subtipos avanzados no aparecen hasta Zona 2).
- Zona 5 (Núcleo Profundo): todos los subtipos disponibles. Dificultad máxima.
- El boss final (ParasitoEnemy) es un Enemy especial no incluido en esta tabla — tiene su propia lógica de fases.